

Documentation Technique : Mémoire des Territoires

Ce document détaille le fonctionnement, l'architecture et les composants techniques de l'application "Mémoire des Territoires". Il est destiné à deux publics : les décideurs (pour une vue d'ensemble) et les développeurs (pour les détails d'implémentation).

1. Vue d'ensemble du projet

Mémoire des Territoires est un générateur automatique de récits sonores. À partir d'un simple thème ou d'une idée (par exemple, "l'ambiance des chantiers navals"), l'application imagine, écrit et produit une expérience audio complète. Elle combine une narration, des voix de personnages, des bruitages et une ambiance sonore pour créer une capsule temporelle immersive, comme un mini-documentaire ou une fiction radiophonique.

2. Ce que fait l'application

L'application transforme une demande utilisateur en un produit audio final à travers plusieurs étapes automatisées :

- **Analyse de la demande** : Elle décompose la requête initiale pour comprendre le sujet principal, les contraintes et les intentions.
- **Génération d'un scénario** : À partir du thème, elle élabore une structure narrative (un plan) puis rédige un véritable scénario avec des dialogues, des descriptions et des événements.
- **Création de l'univers sonore** : L'application sélectionne des sons d'ambiance (ex: le bruit d'un port), des bruitages spécifiques (ex: une sirène de bateau) et choisit des voix pour les personnages du récit.
- **Production finale** : Elle assemble tous ces éléments (narrations, dialogues, sons) sur une ligne temporelle audio pour produire le fichier son final, prêt à être écouté.

3. Stack technique

Voici les technologies principales utilisées dans le projet.

Domaine	Technologie	Rôle
Backend	Python 3.11+	Langage principal pour toute la logique de génération.
	Google Gemini / OpenAI GPT	Modèles de langage pour l'analyse, la structuration et l'écriture.
	FastAPI (probable)	Framework pour exposer la logique via une API (inféré depuis <code>server</code>).
	Pydub / audio-related libs	Manipulation des fichiers audio.
Frontend	React (avec TypeScript)	Framework pour l'interface utilisateur web.
	Vite	Outil de build pour le développement frontend.
Orchestration	Python (<code>orchestrator.py</code>)	Script central qui coordonne les différents agents IA.

Domaine	Technologie	Rôle
Déploiement	Docker	Conteneurisation de l'application pour un déploiement unifié.
Gestion Dépendances	uv / pip	Gestion des paquets Python.
	npm	Gestion des paquets JavaScript pour le frontend.
Automatisation	Makefile	Scripts pour simplifier les tâches courantes (installation, lancement).

4. Architecture du projet

Le projet est organisé autour d'une architecture modulaire basée sur des "agents" spécialisés, coordonnés par un orchestrateur.

- `c:/Users/romua/Documents/Lab_IA/memoiredesterritoires/`
 - `agents/`: **Cœur logique de l'IA**. Chaque sous-dossier est un "agent" spécialisé dans une tâche (analyser, structurer, écrire, produire). C'est ici que la magie opère.
 - `app/`: **Interface utilisateur (Frontend)**. Une application React qui permet aux utilisateurs d'interagir avec le système.
 - `skills/`: **Compétences réutilisables**. Contient des logiques spécifiques (ex: `audio_timeline_composer`) que les agents peuvent utiliser. Cela sépare la décision (agent) de l'action (skill).
 - `orchestrator.py`: **Le chef d'orchestre**. Ce script exécute les agents dans le bon ordre pour passer de la requête initiale au produit final.
 - `config/`: Fichiers de configuration pour les différents modèles d'IA, les étapes du projet, etc. Permet de modifier le comportement sans changer le code.
 - `data/`: Données brutes et sessions de travail. Contient les sources audio, les projets, etc.
 - `output/`: **Dossier de sortie**. C'est ici que les scénarios, structures et fichiers audio finaux sont sauvegardés.
 - `main.py` / `cli.py`: Points d'entrée pour lancer l'application, probablement en mode API ou ligne de commande.
 - `Dockerfile`: Fichier de recette pour construire l'image Docker de l'application.
 - `makefile`: Raccourcis pour les commandes d'installation et de lancement.
 - `requirements.txt` / `pyproject.toml`: Liste des dépendances Python du projet.
 - `package.json`: Liste des dépendances JavaScript pour le frontend.

5. Fonctionnement global

Le flux de travail principal est une chaîne de traitement séquentielle gérée par l'`orchestrator.py`.

1. **Requête** : L'utilisateur soumet une idée via l'interface web (ou un autre point d'entrée).
2. **Orchestration** : `orchestrator.py` reçoit la demande et la transmet au premier agent.
3. **Pipeline d'Agents** :
 - L'**Agent 0** (`request_parser`) analyse et nettoie la demande.
 - Il passe le résultat à l'**Agent 1** (`structure`) qui crée le squelette du récit.

- L'**Agent 2 (writing)** prend ce squelette et rédige le scénario complet.
 - L'**Agent 3 (production)** utilise ce scénario pour assembler la piste audio, en piochant dans les **skills** pour la composition sonore et le choix des voix.
4. **Résultat** : Chaque étape sauvegarde ses productions (structure, scénario, audio) dans le dossier **output/**.

Schéma textuel simplifié du flux :

```
[Requête Utilisateur] -> [Orchestrator]
                        |
                        v
[Agent 0: Analyse] -> [Agent 1: Structure] -> [Agent 2:
Écriture] -> [Agent 3: Production]
                        |
                        v
[Fichier Audio Final]
```

6. Installation et lancement

Ces instructions sont destinées à un développeur souhaitant lancer le projet en local.

Prérequis

- Python 3.11+
- Node.js & npm
- Docker (recommandé)
- **make** (disponible sur Linux/macOS, peut nécessiter une installation sur Windows)

1. Installation

Le **makefile** simplifie l'installation. Exécutez la commande suivante à la racine du projet :

```
# Installe les dépendances Python et JavaScript
make setup
```

Si **make** n'est pas disponible, suivez les étapes manuelles :

```
# Installer les dépendances backend
pip install -r requirements.txt

# Installer les dépendances frontend
cd app
npm install
cd ..
```

2. Lancement du Backend

Le backend est le cœur de l'application qui fait tourner les agents IA.

```
# Lancer le serveur backend (probablement une API FastAPI)
make run-backend
```

Note : La commande exacte peut varier. Consultez le [makefile](#) ou [main.py](#) pour plus de détails.

3. Lancement du Frontend

L'interface web permet d'interagir avec le backend.

```
# Se placer dans le dossier de l'application web
cd app

# Lancer le serveur de développement
npm run dev
```

L'interface sera alors accessible dans votre navigateur, généralement à l'adresse <http://localhost:5173>.

4. Utilisation avec Docker

Le [Dockerfile](#) permet de construire une image contenant toute l'application.

```
# 1. Construire l'image Docker
docker build -t memoire-des-territoires .

# 2. Lancer un conteneur à partir de l'image
docker run -p 8000:8000 -v ./output:/app/output memoire-des-territoires
```

Note : Les ports et volumes peuvent nécessiter des ajustements en fonction de la configuration exacte du serveur dans [main.py](#).